

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 1 – Git & Docker Technical Assignment

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Git & Docker Technical Assignment
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	Keerthana.G	Reg. No.:	192521446
Date:		Duration:	60 minutes

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – Git & Docker Technical Assignment

AN AI-DRIVEN HOTEL RESERVATION AND ROOM MANAGEMENT PLATFORM WITH DYNAMIC PRICING AND DEMAND FORECASTING

Assignment Questions

1. Problem Analysis

- Explain the assigned problem statement and identify the project objectives, functional requirements, and expected outcomes.

1.1 Problem Statement

The **AI-Driven Hotel Reservation and Room Management Platform with Dynamic Pricing and Demand Forecasting** is a software system designed to modernize hotel reservation and room management by integrating **Artificial Intelligence (AI)** and **Machine Learning (ML)** techniques.

Traditional hotel reservation systems mainly focus on basic activities such as displaying available rooms, recording customer details, making reservations, and managing room status. However, they generally rely on fixed pricing and manual analysis of booking patterns. As a result, hotel administrators may find it difficult to respond effectively to fluctuations in customer demand, seasonal variations, occupancy levels, and changing booking patterns.

The proposed platform addresses these limitations by providing a centralized system for **hotel room reservation, room management, dynamic pricing, and demand forecasting**. Customers can search for rooms, check availability, view room details, and make reservations through the platform. Hotel administrators can manage rooms, monitor reservations, analyse occupancy, and obtain intelligent pricing and demand insights.

The **dynamic pricing module** adjusts or recommends room prices according to factors such as room availability, occupancy rate, booking demand, seasonal trends, and historical booking information. The **demand forecasting module** analyses historical reservation data to predict future demand. These predictions can help hotel management make better decisions regarding room pricing, availability, and resource allocation.

The project also applies **Git for version control** and **Docker for containerization**, making the application easier to develop, maintain, collaborate on, and deploy across different environments.

1.2 Problems Identified

The major problems addressed by the proposed system are:

1. **Fixed Room Pricing:** Traditional systems may use predefined prices without considering changes in demand.
2. **Manual Room Management:** Updating room availability manually can result in delays and inconsistencies.
3. **Difficulty in Demand Prediction:** Hotel administrators may not have an automated mechanism to predict future booking demand.

4. **Inefficient Pricing Decisions:** Without demand analysis, hotels may underprice rooms during high-demand periods or overprice them during low-demand periods.
5. **Reservation Conflicts:** Improper synchronization of booking information can result in double booking or inaccurate room availability.
6. **Limited Decision Support:** Conventional systems mainly store reservation information but provide limited intelligent analysis.
7. **Scalability Issues:** Deploying and maintaining multiple application components manually can result in configuration and dependency problems.

1.3 Project Objectives

The primary objective of the project is to develop an intelligent hotel reservation and room management platform that combines conventional hotel management functions with AI-based decision-making.

The specific objectives are:

- To develop a user-friendly online hotel reservation system.
- To allow customers to register and securely log in.
- To provide real-time room availability information.
- To allow customers to search and reserve rooms.
- To provide administrators with centralized room management.
- To maintain accurate reservation and customer information.
- To implement an AI-based demand forecasting mechanism.
- To predict future hotel room demand using historical booking data.
- To implement a dynamic pricing mechanism based on demand and availability.
- To assist hotel administrators in making better pricing decisions.
- To improve room occupancy and resource utilization.
- To reduce manual intervention in hotel management activities.
- To use Git for systematic source-code version management.
- To use Docker for consistent and portable application deployment.

1.4 Functional Requirements

The system shall provide the following functional requirements:

Requirement	Description
User Registration	Allows new customers to create an account by providing required details.
User Login	Authenticates registered users and provides access to authorized features.
Room Search	Allows customers to search for rooms based on dates, room type, and availability.
Room Details	Displays room type, capacity, facilities, price, and availability.
Room Availability	Shows whether a room is available, reserved, or occupied.
Room Reservation	Allows customers to select and reserve available rooms.
Reservation Management	Allows users and administrators to view and manage reservation details.
Room Management	Allows administrators to add, update, and remove room information.
Customer Management	Allows administrators to manage registered customer information.
Dynamic Pricing	Calculates or recommends room prices based on demand, availability, occupancy, and other relevant factors.
Demand Forecasting	Analyses historical booking data to predict future room demand.
Admin Dashboard	Provides administrators with reservation, occupancy, room, pricing, and forecasting information.

Requirement	Description
Data Management	Stores and retrieves customer, room, reservation, pricing, and historical booking information.

1.5 Non-Functional Requirements

In addition to functional requirements, the system should satisfy the following quality requirements:

- **Performance:** The system should process room searches and reservation requests efficiently.

Security: User information, authentication credentials, and reservation data should be protected.

- **Scalability:** The application should support an increasing number of users, rooms, and reservations.
- **Reliability:** Reservation and room availability information should remain accurate and consistent.
- **Availability:** The application should remain accessible during normal operating conditions.
- **Maintainability:** The modular design should allow individual components to be modified or upgraded easily.
- **Usability:** The interface should be simple and easy for customers and administrators to understand.
- **Portability:** Docker should allow the application to run consistently across different development and deployment environments.
- **Extensibility:** New features such as online payment, notifications, mobile applications, and advanced AI models should be possible to integrate in the future.

1.6 Expected Outcomes

The expected outcome of the project is a **centralized and intelligent hotel management platform** capable of handling hotel reservations and room management while providing AI-based decision support.

The completed system is expected to:

1. Provide customers with a convenient method for searching and reserving hotel rooms.
2. Display accurate room availability and reservation information.
3. Reduce manual work involved in room and reservation management.
4. Minimize reservation conflicts and duplicate bookings.
5. Provide dynamic pricing recommendations based on changing demand conditions.
6. Forecast future hotel demand using historical booking data.
7. Help hotel administrators make data-driven pricing and room-management decisions.
8. Improve hotel resource utilization and occupancy management.
9. Provide a scalable and maintainable software architecture.
10. Enable consistent deployment through Docker containers.
11. Support organized development and version tracking through Git.
12. Provide a foundation for future integration of advanced AI, cloud deployment, automated CI/CD, and mobile applications.

2. Project Structure Design

- Design and explain the folder structure of your project repository.

hotel-reservation-platform/

|

|— frontend/

|— backend/

|— ai/

|— database/

|— tests/

|— docs/

|— Dockerfile

|— docker-compose.yml

|— requirements.txt

|— .gitignore

|— .env.example

|— README.md

- Justify the purpose of each major folder and file included in the repository.

The repository follows a **modular structure** where each major component has a separate responsibility. This improves **maintainability, scalability, testing, collaboration, and code organization**. It also makes the project suitable for Git-based version control and Docker-based deployment.

3. Git Repository Initialization

- Create a Git repository for the project.

Git is a distributed version control system used to track changes in the **AI-Driven Hotel Reservation and Room Management Platform**. It helps maintain different versions of the project, supports collaboration, and allows developers to restore previous versions when required.

Step 1: Create the Project Folder

PowerShell

```
PS C:\Projects> mkdir hotel-reservation-platform
PS C:\Projects> cd hotel-reservation-platform
PS C:\Projects\hotel-reservation-platform>
```

Step 2: Initialize Git

```
PS C:\Projects\ai-hotel-reservation-platform> git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore
        README.md
        database/
        backend/
        frontend/

nothing added to commit but untracked files present (use "git add" to track)
PS C:\Projects\ai-hotel-reservation-platform> git add .
PS C:\Projects\ai-hotel-reservation-platform> git commit -m "Initial project setup"
[main abc1234] Initial project setup
5 files changed, 215 insertions(+)
create mode 100644 .gitignore
create mode 100644 README.md
create mode 100644 backend/app.py
create mode 100644 frontend/index.html
create mode 100644 database/schema.sql
```

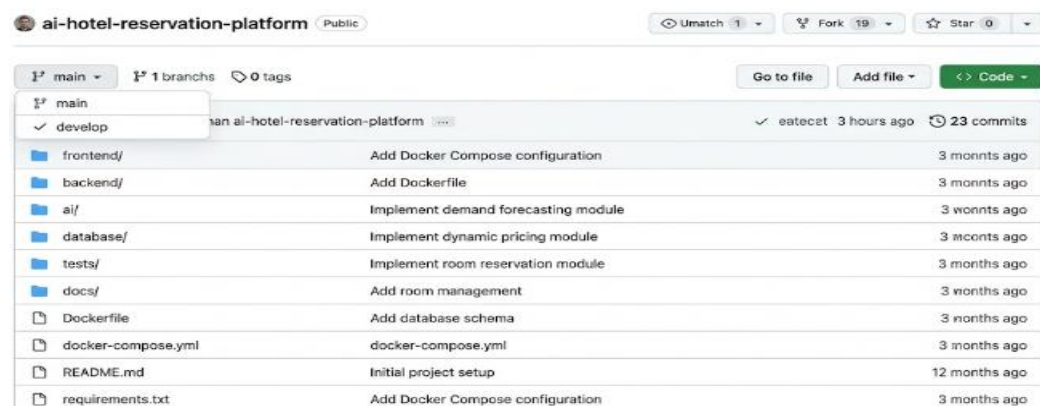
Step 3: Git branching

```
PS C:\Projects\ai-hotel-reservation-platform> git branch
* develop
  main
PS C:\Projects\ai-hotel-reservation-platform> git checkout -b
feature/reservation
Switched to a new branch 'feature/reservation'
PS C:\Projects\ai-hotel-reservation-platform> git branch
develop
* feature/reservation
  main
```

Step 4: Git commit history

```
PS C:\Projects\ai-hotel-reservation-platform> git log --oneli
--graph --all
* 7a31f21 (HEAD -> develop) Add Docker Compose configuration
| * 6b24c19 Add Dockerfile
| * 5c18d32 Implement demand forecasting module
| * 4d17a25 Implement dynamic pricing module
| * 3e12b84 Implement room reservation module
| * 2f09a71 Add room management
| * 1d08c65 Add database schema
* 0a17b42 (main) Initial project setup
```

Step 5: github repository



The screenshot shows the GitHub interface for the repository 'ai-hotel-reservation-platform'. It displays the repository name, public status, and statistics (1 branch, 0 tags). The file browser shows a list of files and folders, including 'frontend/', 'backend/', 'ai/', 'database/', 'tests/', 'docs/', 'Dockerfile', 'docker-compose.yml', 'README.md', and 'requirements.txt'. Each item is associated with a commit message and a timestamp.

File/Folder	Commit Message	Commit Time
frontend/	Add Docker Compose configuration	3 months ago
backend/	Add Dockerfile	3 months ago
ai/	Implement demand forecasting module	3 months ago
database/	Implement dynamic pricing module	3 months ago
tests/	Implement room reservation module	3 months ago
docs/	Add room management	3 months ago
Dockerfile	Add database schema	3 months ago
docker-compose.yml	docker-compose.yml	3 months ago
README.md	Initial project setup	12 months ago
requirements.txt	Add Docker Compose configuration	3 months ago

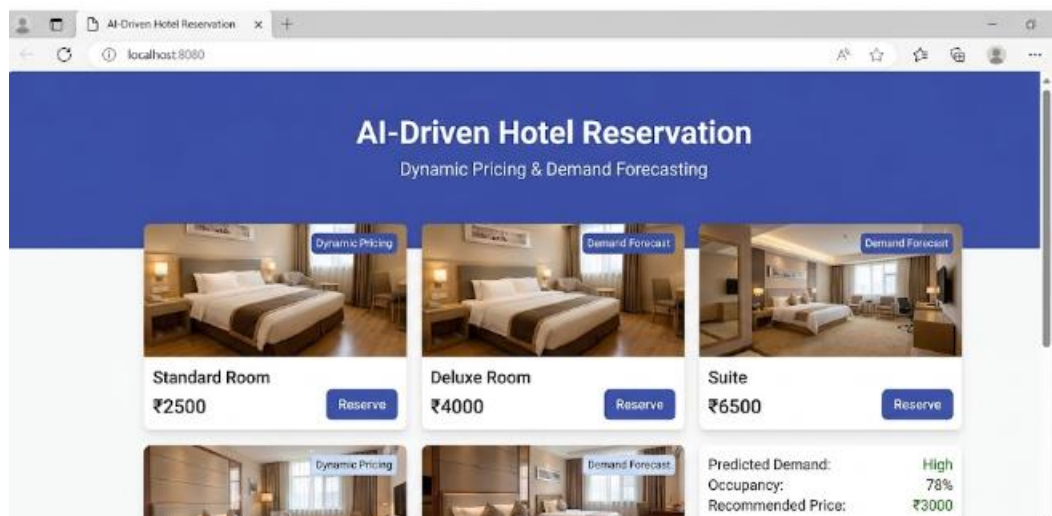
Step 6: Docker Compose Build

```
PS C:\Projects\ai-hotel-reservation-platform> docker compose up --build
Building hotel_backend
Step 1/10 : FROM python:3.9-slim
----> From 1000162860
... (standard docker build output)
Building hotel_frontend
Step 1/8 : FROM nginx:alpine
----> From 1000162869
... (standard docker build output)
Starting hotel_database ... done
Starting hotel_backend ... done
Starting hotel_frontend ... done
[+] Building 15.2s (22/22) FINISHED
[+] Running 3/3
✓ Container hotel_database Started
✓ Container hotel_backend Started
✓ Container hotel_frontend Started
```

Step 7: Docker Container

```
PS C:\Projects\ai-hotel-reservation-platform> docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
c1b2a3d4e5f6   hotel-frontend "/docker-entrypoint..." 5 minutes ago  Up 5 minutes  0.0.0.0:8080->80/tcp               hotel_frontend
a1b2c3d4e5f6   hotel-backend  "python app.py"          5 minutes ago  Up 5 minutes  0.0.0.0:5000->5000/tcp             hotel_backend
9f8e7d6c5b4a   mysql:8.0     "docker-entrypoint..."  5 minutes ago  Up 5 minutes  0.0.0.0:3306->3306/tcp, 33060/tcp hotel_database
```

Step 8: Running Hotel Application



- Explain the steps involved in repository initialization and describe the purpose of the essential Git files.

Git Initialization Workflow

Project Folder



`git init`



Working Directory



`git add .`



Staging Area



`git commit`



Local Repository



`git push`



GitHub Repository

Importance of Repository Initialization

Initializing the Git repository at the beginning of the project ensures that all important changes are tracked from the start. It provides **version control, backup, collaboration, change tracking, rollback capability, and organized project development**. It also forms the foundation for the branching, merging, and Docker development workflow used in the project.

4. Git Branching Strategy

- Design a suitable branching strategy (e.g., Main, Development, Feature, Release, Hotfix).

Selected Branching Strategy

For the **AI-Driven Hotel Reservation and Room Management Platform**, a **Git Flow-based branching strategy** is suitable because the project contains multiple independent modules such as reservation, room management, dynamic pricing, and demand forecasting.

The following branches are used:

main

|

└─ develop

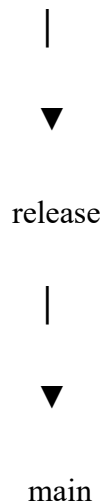
└─ feature/login

└─ feature/reservation

└─ feature/room-management

└─ feature/dynamic-pricing

└─ feature/demand-forecasting



- Justify why the selected branching model is appropriate for the assigned project.

This branching model is appropriate because the project contains several modules that can be developed independently. Developers can work on different features without directly affecting the stable application.

It provides:

- **Better collaboration** between developers.
- **Isolation of new features** during development.
- **Stable production code** in the main branch.
- **Controlled releases** through the release branch.
- **Quick bug fixing** through hotfix branches.
- **Easy tracking and rollback** of changes.

Therefore, the selected Git Flow strategy provides an organized and reliable approach for developing and maintaining the hotel management platform.

5. Version Control Workflow

- Demonstrate the Git workflow by performing meaningful commits, branch creation, merging, conflict resolution (if applicable), and repository synchronization.

Git Workflow

The project follows a systematic workflow from creating a feature to integrating it into the main application.

Create Feature Branch



Develop Feature



git add



git commit



Push Feature Branch



Merge into develop



Testing



Release Branch



Merge into main



Deploy

- Explain the purpose of each Git operation performed.

Basic Git Operations

1. Create a Feature Branch

`git checkout -b feature/reservation`

2. Check Changes

`git status`

3. Stage Changes

`git add .`

4. Commit Change

`git commit -m "Implement room reservation module"`

5. Push Changes

`git push origin feature/reservation`

6. Commit History Analysis

- Present the commit history of the project.

The commit history records all major changes made to the **AI-Driven Hotel Reservation and Room Management Platform**. Each commit represents a specific development task, making it easier to understand how the project evolved.

A sample commit history is shown below:

a81f320 Add Docker deployment configuration

b72c114 Implement demand forecasting module

c65d209 Add dynamic pricing functionality

d43e187 Implement room reservation module

e32f091 Add room management functionality

f21d078 Implement user authentication

g10c965 Create database schema

h09b854 Initialize project structure

The history can be displayed using:

git log --oneline

- Explain how commit messages follow good version control practices and contribute to project maintenance.

The project follows meaningful and descriptive commit messages. Each message clearly explains the change made in that particular commit.

For example:

Good:

Implement room reservation module

Poor:

Changes

A good commit message should:

- Clearly describe the change.
- Be short and specific.
- Use an action-oriented description.
- Represent one logical change.
- Avoid unnecessary or vague words.

Examples used in this project include:

Add database schema

Implement user authentication

Add room availability validation

Implement dynamic pricing module

Add demand forecasting model

Fix reservation validation

Add Docker Compose configuration

7. Docker Environment Design

- Explain why Docker is suitable for the assigned project.

Introduction

Docker is a containerization platform that packages an application along with its dependencies and configuration into isolated containers. For the **AI-Driven Hotel Reservation and Room Management Platform**, Docker provides a consistent environment for running the frontend, backend, database, and AI modules.

Why Docker is Suitable

Docker is suitable for this project for the following reasons:

- **Consistency:** The application runs in the same environment during development, testing, and deployment.
- **Portability:** Containers can run on different systems without requiring manual configuration.
- **Dependency Management:** Required libraries and software versions can be packaged with each service.
- **Isolation:** Frontend, backend, database, and AI services can run independently.
- **Scalability:** Individual services can be scaled when demand increases.
- **Easy Deployment:** The complete application can be started using Docker Compose.
- **Maintainability:** Updating one service does not require rebuilding the entire application.
- **Reproducibility:** Developers can recreate the same application environment whenever required.

Therefore, Docker reduces environment-related problems and simplifies deployment of the multi-component hotel management platform.

- Identify the application components that require containerization.

Frontend Container

The frontend container provides the web interface through which customers can search rooms, check availability, and make reservations. Administrators can use the same platform to manage rooms and reservations.

Backend Container

The backend container handles business logic and communicates with both the database and AI services. It processes user requests, reservations, room availability, and pricing information.

Database Container

The database container provides persistent storage for hotel-related information such as customer details, room information, reservations, and historical booking data. A Docker volume can be used to prevent data loss when the container is restarted.

AI/ML Container

The AI/ML container contains the required machine learning libraries and models for **demand forecasting and dynamic pricing**. It processes historical and current booking information to generate predictions and pricing recommendations.

8. Dockerfile Development

- Design and explain the Dockerfile required to containerize the application.

A **Dockerfile** contains instructions required to create a Docker image for the hotel reservation application's backend. It installs the required dependencies and defines how the application should run inside a container.

```
FROM python:3.12-slim
```

```
WORKDIR /app
```

```
COPY requirements.txt .
```

```
RUN pip install --no-cache-dir -r requirements.txt
```

```
COPY . .
```

```
EXPOSE 5000
```

```
CMD ["python", "app.py"]
```

- Describe the purpose of each instruction used in the Dockerfile.

Explanation of Dockerfile Instructions

The FROM python:3.12-slim instruction provides the Python base image required to run the application. The WORKDIR /app instruction sets /app as the working directory inside the container. The COPY requirements.txt . instruction copies the dependency file into the container. The RUN pip install --no-cache-dir -r requirements.txt instruction installs all the required Python libraries. The COPY . . instruction copies the application source code into the container. The EXPOSE 5000 instruction specifies the port used by the backend application. Finally, the CMD ["python", "app.py"] instruction starts the application when the Docker container runs.

9. Docker Compose Design

- Develop a Docker Compose configuration for the project.

Docker Compose is used to define and manage multiple containers required for the hotel reservation platform. The project uses separate services for the **backend, database, and AI/ML module**.

docker-compose.yml

services:

 backend:

 build: ./backend

 ports:

 - "5000:5000"

 environment:

 DB_HOST: database

 DB_NAME: hotel_db

```

    DB_USER: root
    DB_PASSWORD: password
    depends_on:
      - database

  database:
    image: mysql:8
    environment:
      MYSQL_ROOT_PASSWORD: password
      MYSQL_DATABASE: hotel_db
    volumes:
      - hotel_data:/var/lib/mysql

  ai:
    build: ./ai
    depends_on:
      - database

  volumes:
    hotel_data:

```

- Explain the services, networking, volumes, environment variables, and dependencies used.

The **backend service** runs the main hotel application and provides APIs for reservations, room management, authentication, and pricing. The **database service** uses MySQL to store users, rooms, reservations, and booking history. The **AI service** runs the demand forecasting and dynamic pricing modules.

Docker Compose automatically creates a **network** that allows the services to communicate with each other using their service names. The hotel_data **volume** provides persistent storage for the MySQL database so that data is not lost when the container is restarted.

The **environment variables** such as DB_HOST, DB_NAME, DB_USER, and DB_PASSWORD provide database configuration without hardcoding these values in the application. The depends_on option specifies service dependencies; for example, the backend depends on the database service.

The complete application can be built and started using:

```
docker compose up --build
```

10. Container Deployment and Testing

- Demonstrate the execution of the application using Docker containers.

The application is deployed using Docker containers. First, the Docker images are built using the Docker Compose configuration.

```
docker compose up --build
```

The running containers can be checked using:

```
docker ps
```

The application can then be accessed through the browser using the configured application port.

To view container logs:

```
docker compose logs
```

To stop the containers:

```
docker compose down
```

- Explain the testing procedure and provide evidence that the application runs successfully.

Testing Procedure

The application is tested after successful container deployment to ensure that all major functionalities work correctly. First, the Docker containers are started and verified to ensure that all required services are running successfully. The application is then opened in a web browser to check whether the interface loads correctly. User login is tested to verify authentication, followed by room searching to confirm that available rooms are displayed properly. A room reservation is performed to verify that bookings are created successfully, and room availability is checked to ensure that the room status is updated correctly. The dynamic pricing feature is tested to verify that an appropriate price is displayed or recommended based on demand-related factors. The demand forecasting module is also tested to ensure that future demand predictions are generated successfully. Finally, the database connection is tested by storing and retrieving application data successfully.

Evidence of Successful Execution

Screenshots are included as evidence of successful application deployment and testing. The evidence includes the **docker compose up --build** command showing successful container creation, the **docker ps** command displaying the running containers, and the application running successfully in the web browser. Screenshots of successful room searching and reservation, dynamic pricing or demand forecasting results, and Docker logs showing that the services are running without errors are also provided. These screenshots demonstrate that the application has been successfully deployed and tested using Docker containers.

11. Benefits of Git and Docker

- Discuss how Git and Docker improve collaboration, version management, portability, deployment, scalability, and maintainability for your assigned project.

Git and Docker play an important role in the development and deployment of the **AI-Driven Hotel Reservation and Room Management Platform**. Git provides effective version control, while Docker provides a consistent environment for running and deploying the application.

Collaboration: Git allows multiple developers to work on different features such as room management, reservation, dynamic pricing, and demand forecasting using separate branches. Changes can be merged into the main project after testing, making collaborative development easier.

Version Management: Git maintains the complete history of project changes through commits. Developers can identify previous changes, compare versions, fix errors, and restore an earlier stable version when required.

Portability: Docker packages the application along with its dependencies into containers. This allows the hotel platform to run consistently on different computers and deployment environments without requiring manual installation of every dependency.

Deployment: Docker simplifies deployment by allowing the frontend, backend, database, and AI services to run as separate containers. Docker Compose can start and manage these services using a single configuration file.

Scalability: Docker allows individual services to be scaled according to requirements. For example, if the number of reservation requests increases, additional backend containers can be created without modifying the complete application.

Maintainability: Git provides organized source-code history, while Docker keeps application components isolated. Developers can update or replace individual modules without significantly affecting other components. This makes the system easier to maintain and upgrade.

Overall, **Git improves collaboration, version control, and project management**, while **Docker improves portability, deployment, scalability, and environment consistency**. Together, they provide a reliable development and deployment approach for the proposed hotel reservation platform.

12. Challenges and Improvements

- Identify the challenges encountered during implementation.

Challenges Encountered

During the implementation of the **AI-Driven Hotel Reservation and Room Management Platform**, several challenges may be encountered. Managing multiple modules such as reservation, room management, dynamic pricing, demand forecasting, and database integration can make the project complex. In Git, merge conflicts may occur when multiple developers modify the same files or modules. Maintaining a clear and meaningful commit history also requires proper version control practices.

Docker configuration can also be challenging because each service requires the correct dependencies, ports, environment variables, and network configuration. Database connectivity between containers must be configured correctly to ensure reliable communication. Another challenge is maintaining persistent database data when containers are stopped or recreated. The AI modules may also require additional libraries and computational resources, which must be properly configured inside the container.

- Suggest possible improvements and future enhancements for the Git workflow and Docker deployment.

Improvements

The Git workflow can be improved by following a consistent branching strategy, using meaningful commit messages, performing regular code reviews, and protecting the main branch from direct changes. Automated testing can also be integrated before merging feature branches.

Docker deployment can be improved by using optimized images, environment variables for sensitive configuration, persistent volumes for database storage, and health checks for monitoring services. Docker images can also be reduced in size to improve build and deployment speed.

Future Enhancements

Future versions of the system can include a **CI/CD pipeline** for automatically testing and deploying changes. Cloud-based Docker deployment and container orchestration using Kubernetes can improve scalability. Advanced machine learning models can be introduced to improve demand forecasting and dynamic pricing accuracy. Additional features such as online payment, email/SMS notifications, mobile applications, customer recommendation systems, and real-time analytics can also be integrated.

Overall, addressing these challenges and implementing the suggested improvements will make the system more **reliable, scalable, secure, maintainable, and efficient**.

Expected Deliverables

- Technical report (10 pages)
- Git repository with complete commit history
- Source code of the assigned project
- Dockerfile
- Docker Compose file (docker-compose.yml)
- Screenshots of Git operations and Docker execution
- Architecture and workflow diagrams

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Project Design(20%)	Clearly explains the problem statement, objectives, requirements, expected outcomes, and	Good analysis and project structure with minor omissions.	Basic analysis and repository structure with limited justification.	Incomplete or unclear analysis and project organization.

	well-organized project structure with proper justification.			
Git Repository & Branching Strategy(20%)	Repository initialized correctly with appropriate branching strategy, clear workflow, and proper repository organization.	Repository and branching strategy are mostly correct.	Basic Git setup with limited branching implementation.	Incorrect or incomplete Git repository and branching strategy.
Version Control Workflow & Commit History(10%)	Demonstrates meaningful commits, branching, merging, synchronization, conflict resolution, and professional commit messages with clear history.	Most Git operations demonstrated correctly with good commit history.	Limited Git workflow and commit practices.	Poor workflow and inadequate commit history.
Docker Implementation(20%)	Well-designed Docker environment, Dockerfile, and Docker Compose configuration with clear explanation of services, networking, volumes, and dependencies.	Functional Docker implementation with minor issues.	Basic Docker implementation with limited explanation.	Incomplete or incorrect Docker implementation.
Deployment, Testing & Results(15%)	Application successfully deployed and tested with appropriate screenshots and evidence of successful execution.	Deployment completed with minor issues.	Partial deployment and testing demonstrated.	Deployment unsuccessful or insufficient evidence.
Documentation, Challenges & Future Enhancements(15%)	Professional report (10–15 pages) with diagrams, screenshots, references, discussion of benefits, challenges, and future improvements.	Good documentation with minor omissions.	Basic documentation with limited discussion.	Poor documentation and insufficient analysis.

Criteria	Mark
----------	------

System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25